

Meta Learning methods

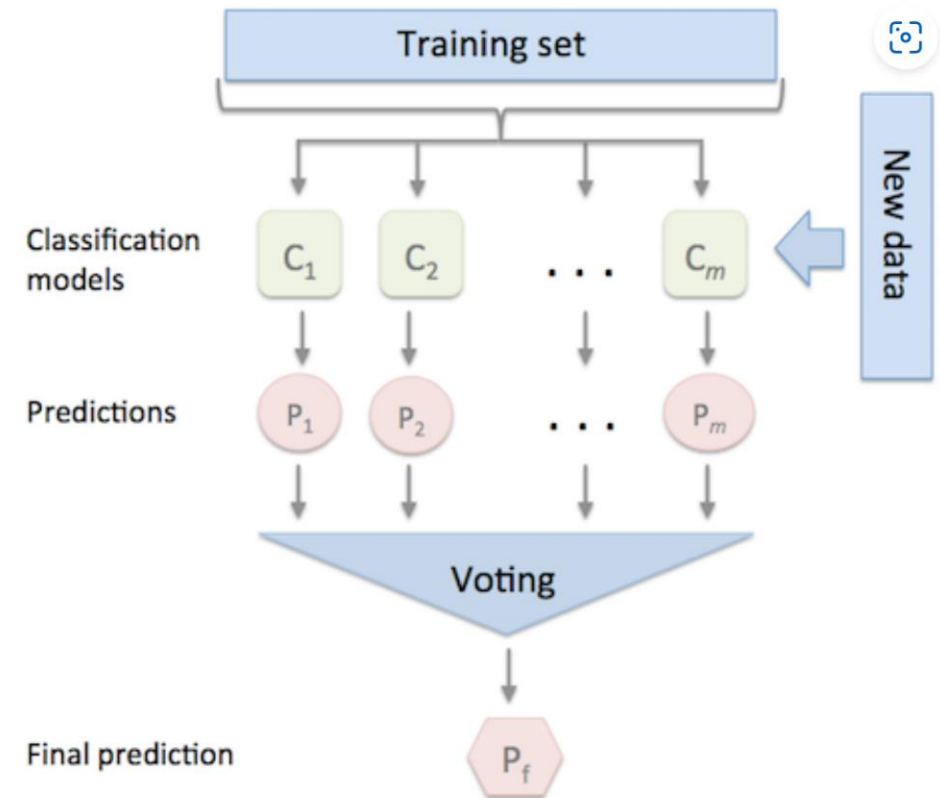
25-11-2024

Caroline König

Meta learning

Meta learning refers to learning algorithms that learn from other learning algorithms.

Combining several *weak learners* to give a strong learner.



Meta learning - Advantages

1. Higher accuracy
 - We trade computation time for classifier weakness.
 - Netflix competition (2009) won by a combination of 107 hybrid classifiers.
 - More: Most of the top teams were multi-classifiers.
2. Combine strengths of different classifiers.
3. Avoid overfitting.

Ensemble learning

- **Ensemble learning** involves combining multiple models to improve the overall performance of the prediction. The idea is that by leveraging the strengths of various models, the ensemble can achieve better results than any single model could on its own.
- **Example:** A voting system where several different machine learning models (like decision trees, logistic regression, and support vector machines) are trained on the same dataset. The final prediction is made based on the majority vote or averaging their outputs.

Condorcet's jury theorem

Addresses the decision-making process of a group of individuals and the probability of that group making a correct decision.

The probability that the majority of the n jurors make the correct decision increases with n if $p > 0.5$. As n becomes large, the probability that the majority decision is correct approaches 1.

Used in ensemble learning: Combining the outputs using majority vote have probability of success p' such that $p' > p$.

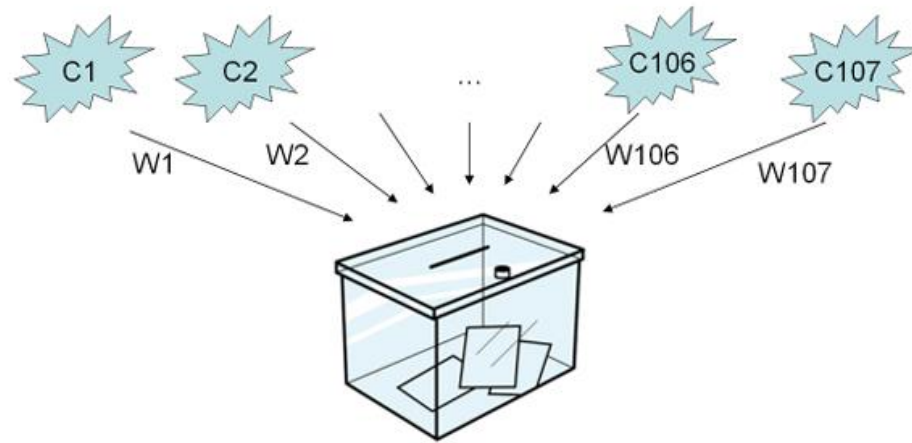
Example: 3 classifiers c_1, c_2, c_3 with $p_1 = 0.7, p_2 = 0.85, p_3 = 0.75$

Combining weak learners

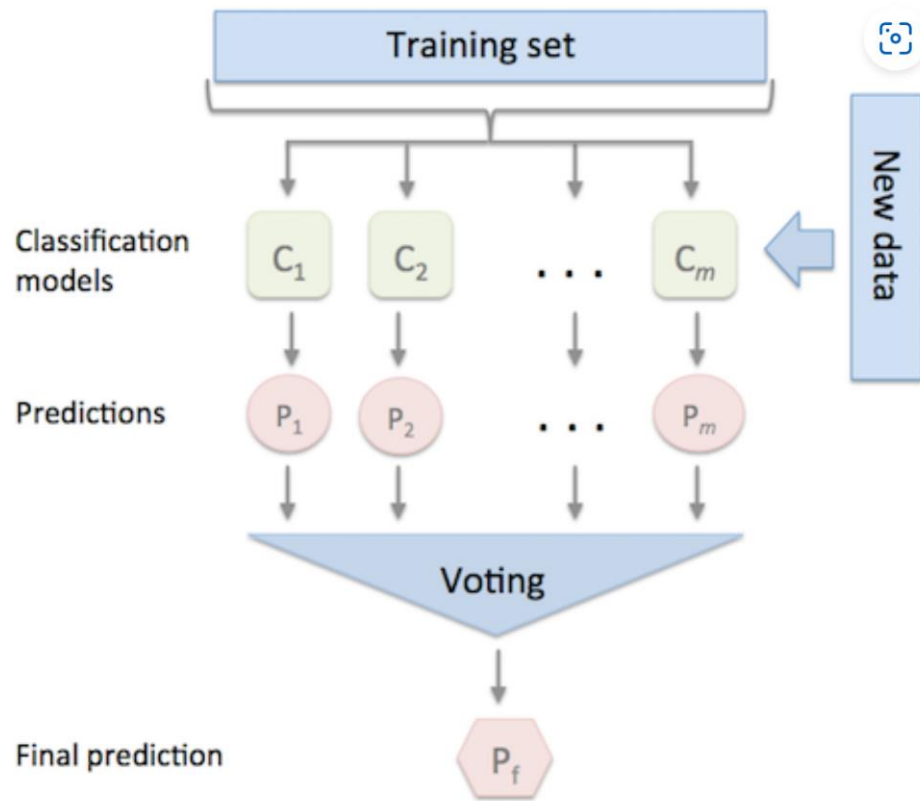
- Voting
 - Each weak learner votes, and votes are combined.
- Experts that abstain
 - A weak learner only counts when it's expert on this kind of instances.
 - Otherwise it abstains (or goes to sleep).

Voting

- How to combine votes?
 - Majority voting scheme.
 - Weights depending on prediction errors.
 - Weights depending on confidences.
 - Maximizing diversity.



Voting schemes



Majority voting

Predicts the class label y via majority voting for each classifier C_i :

$$\hat{y} = \text{mode}\{C_1(\mathbf{x}), C_2(\mathbf{x}), \dots, C_m(\mathbf{x})\}$$

Weighted Majority voting

Calculates a weighted majority vote by associating a weight w_j with each classifier:

$$\hat{y} = \arg \max_i \sum_{j=1}^m w_j \chi_A(C_j(\mathbf{x}) = i),$$

Example

- classifier 1 -> class 0
- classifier 2 -> class 0
- classifier 3 -> class 1
- Majority voting: prediction to class 0
- Weighted majority voting: assigning class weights {0.2, 0.2, 0.6}

$$\arg \max_i [0.2 \times i_0 + 0.2 \times i_0 + 0.6 \times i_1] = 1$$

Weighted majority (Littlestone-Warmuth 92)

```
initialize classifiers  $c_1 \dots c_N$  with weight  $w_i = 1/N$  each;  
for each example  $x$  in sequence do  
    collect predictions  $c_1(x) \dots c_N(x)$ ;  
    prediction( $x$ ) = sign[  $w_1 c_1(x) + \dots + w_N c_N(x) - 1/2$  ]  
    get true label  $y$  for  $x$ ;  
    for each  $i=1..N$ ,  
        if ( $c_i(x) \neq y$ ) then  $w_i = w_i/2$ ;  
renormalize weights to sum 1;
```

Weighted majority (Littlestone-Warmuth 92)

- Weights depend exponentially on error
- At least as good as best weak learner in time $O(\log N)$
- Often much better; more when classifiers are uncorrelated.
- Good for online prediction and when many classifiers

Soft voting

- Prediction based on the probabilities for each classifier: $\hat{y} = \arg \max_i \sum_{j=1}^m w_j p_{ij},$

- $C_1(\mathbf{x}) \rightarrow [0.9, 0.1]$
- $C_2(\mathbf{x}) \rightarrow [0.8, 0.2]$
- $C_3(\mathbf{x}) \rightarrow [0.4, 0.6]$

Using uniform weights, we compute the average probabilities:

$$p(i_0 | \mathbf{x}) = \frac{0.9 + 0.8 + 0.4}{3} = 0.7$$

$$p(i_1 | \mathbf{x}) = \frac{0.1 + 0.2 + 0.6}{3} = 0.3$$

$$\hat{y} = \arg \max_i [p(i_0 | \mathbf{x}), p(i_1 | \mathbf{x})] = 0$$

However, assigning the weights $\{0.1, 0.1, 0.8\}$ would yield a prediction $\hat{y} = 1$:

$$p(i_0 | \mathbf{x}) = 0.1 \times 0.9 + 0.1 \times 0.8 + 0.8 \times 0.4 = 0.49$$

$$p(i_1 | \mathbf{x}) = 0.1 \times 0.1 + 0.2 \times 0.1 + 0.8 \times 0.6 = 0.51$$

$$\hat{y} = \arg \max_i [p(i_0 | \mathbf{x}), p(i_1 | \mathbf{x})] = 1$$

Ensemble learning

1. Stacking
2. Bagging (Bootstrap aggregating)
3. Boosting

Stacking (Stacked Generalization)

Stacking involves training multiple models and then using another model (meta-learner) to combine their predictions. The base models make predictions, which are then used as input features for the meta-learner.

Example: Combining logistic regression, decision trees, and neural networks, with another model like a linear regression or another logistic regression as the meta-learner.

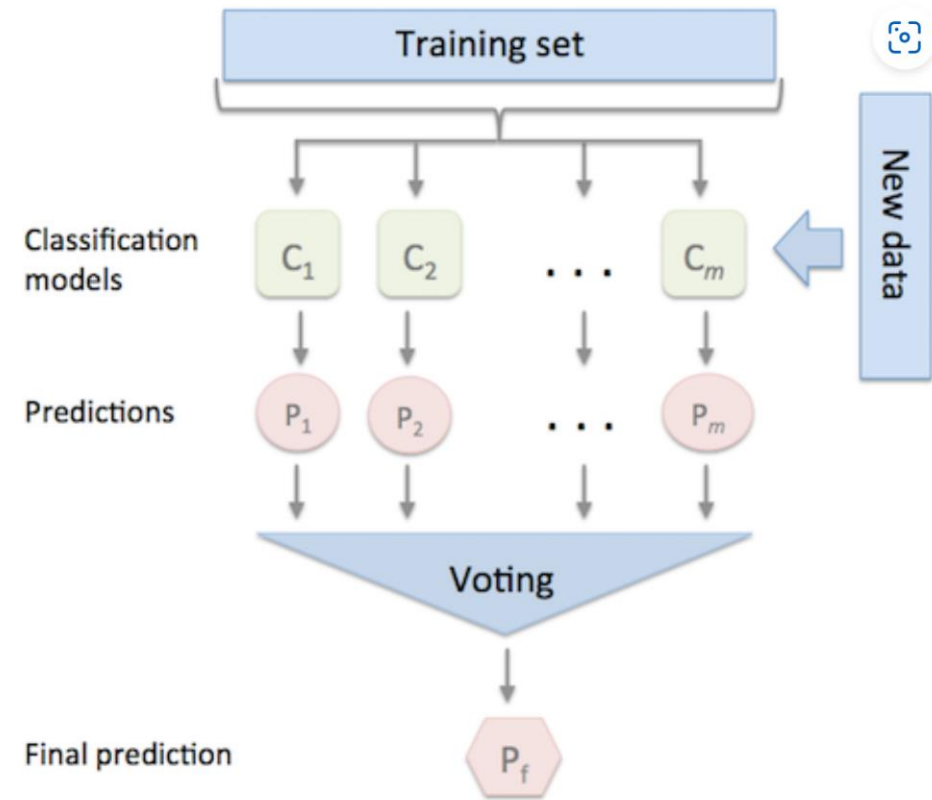
Stacking

The most widely known meta-learning algorithm is called *stacked generalization*, or stacking for short.

“In order to induce a meta classifier, first the base classifiers are trained (stage one), and then the Meta classifier (second stage). In the prediction phase, base classifiers will output their classifications, and then the Meta-classifier(s) will make the final classification (as a function of the base classifiers).”

Stacking (Stacked Generalization)

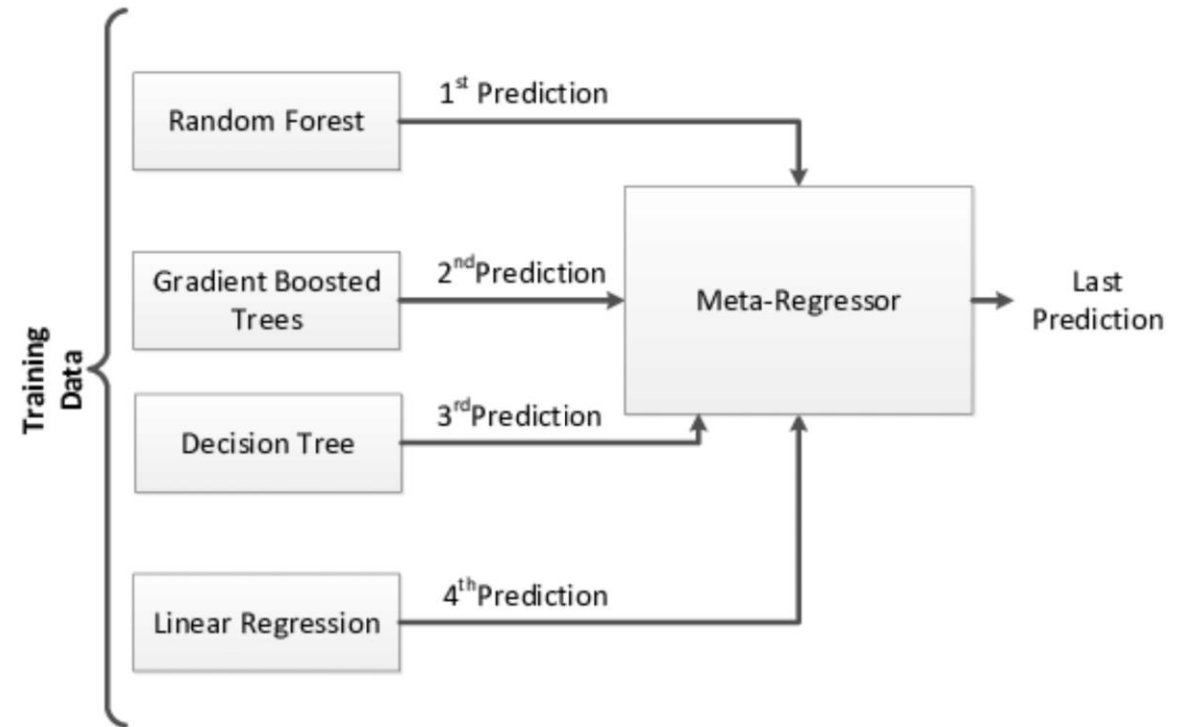
The meta-model learns how to best combine the predictions of the base models by weighting the combination of predictions.



Stacking (Stacked Generalization)

Improves diversity by combining the predictions of multiple models.

Different models with different strengths and weaknesses.



Bagging (Bootstrap Aggregating)

Bagging involves training multiple models (usually the same type) on different subsets of the training data and then aggregating their predictions. The subsets are created by bootstrapping (sampling with replacement).

Example: Random Forest, which combines multiple decision trees.

Bagging (Bootstrap Aggregating)

To reduce the variance of an estimator, it is helpful to average estimates from independent draws from the data.

The idea of bagging is to combine the predictions of a high-variance predictor trained on independent bootstrap samples from the same dataset, to make the combined predictions more robust (i.e. with lower variance) and, therefore, more accurate.

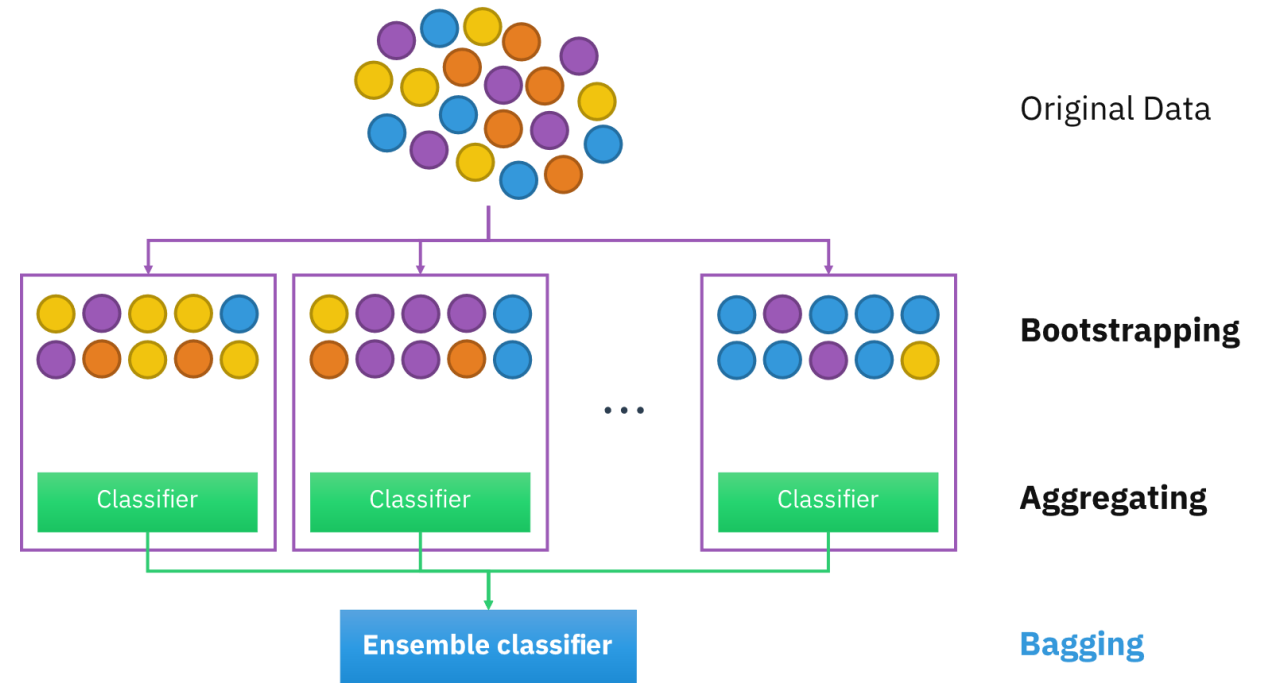
Trees typically suffer from high variance (= overfitting) so it is specially useful in Decision trees (high variance or sensibility to training data set).

Bagging (Breiman 96)

1. Get a dataset S of N labeled examples on A attributes;
2. Build N bagging replicas of S : $S_1, \dots, S_N$;
 1. S_i = draw N samples from S with replacement;
3. Use the N replicas to build N weak learners $C_1, \dots, C_N$;
4. Predict using majority vote of the C_i s

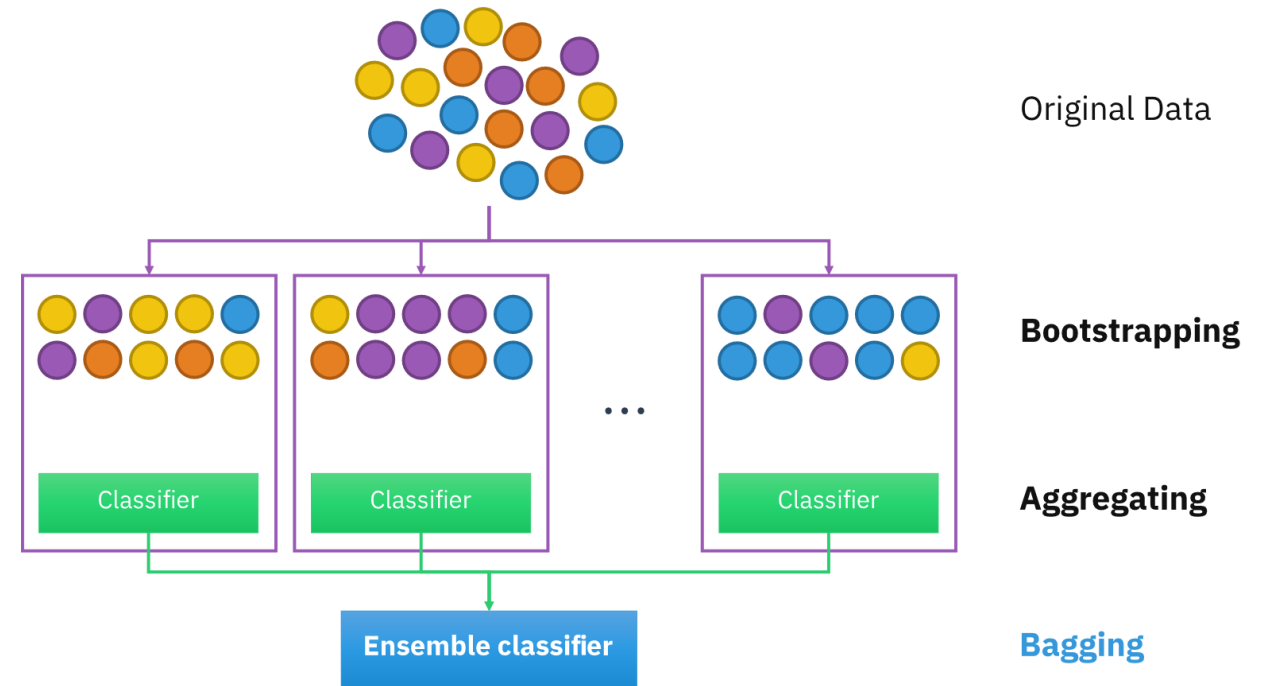
Bootstrap Aggregation(Bagging)

- Generates m datasets for M weak classifiers with bootstrapping.
- The M models are trained on the m datasets.



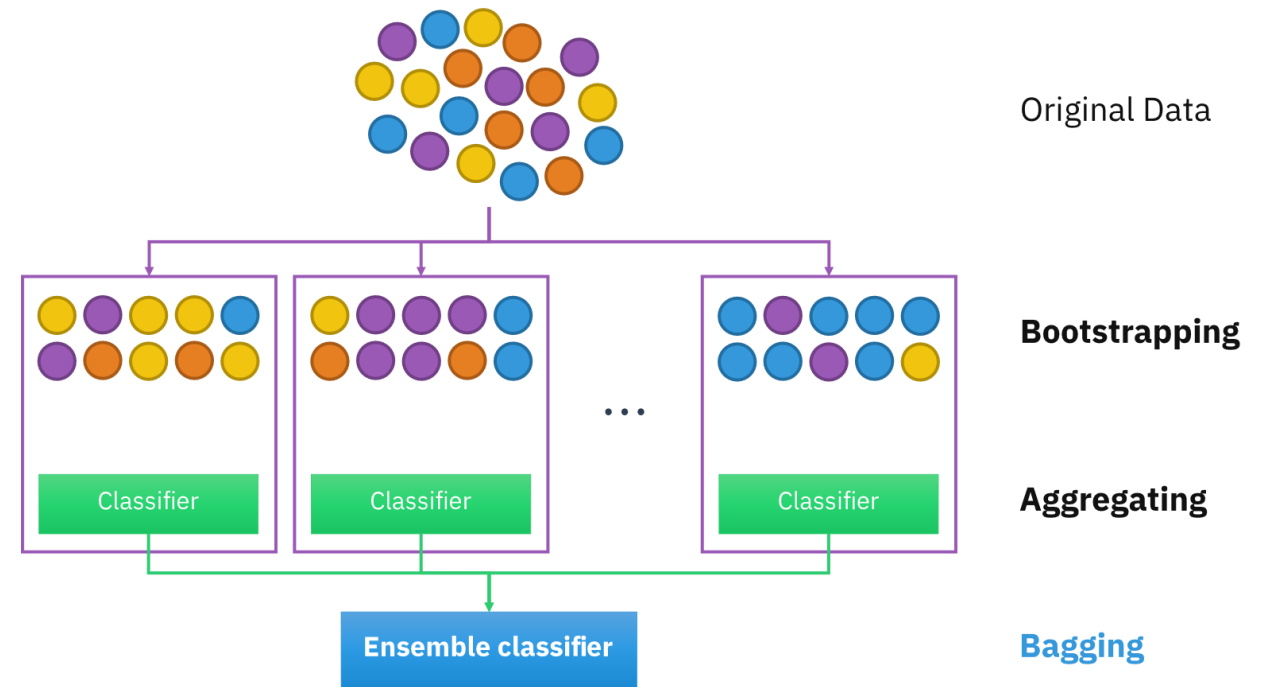
Bootstrap Aggregation(Bagging)

- Regression: averages the outcome of the M models.
- Classification: uses a voting scheme.



Bootstrap Aggregation(Bagging)

- Voting scheme: method of combining the predictions of multiple classifiers to produce a single prediction.
- Soft versus hard voting.



Separating data

Example of building training sets:

Original:	1	2	3	4	5	6	7	8
Training Set1:	2	7	8	3	7	6	3	1
Training Set2:	7	8	5	6	4	2	7	1
Training Set3:	3	6	2	7	5	6	2	2
Training Set4:	4	5	1	4	6	4	3	8

Any samples that are not chosen for the bootstrapped dataset are placed in a separate dataset called the out-of-bag dataset (OOB)

Out-of-bag (OOB) error

The **OOB error** is an estimation of generalization error that can be used as validation error to select appropriate values for the hyperparameters.

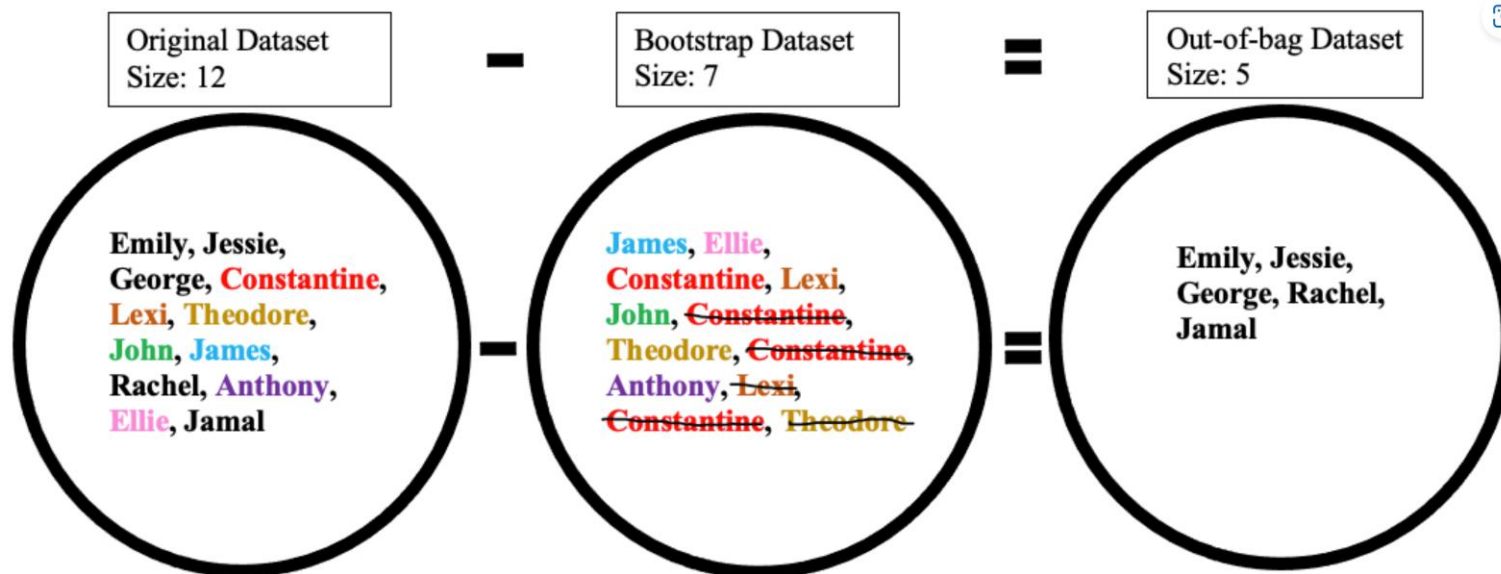
As a direct consequence, **there is no need for cross-validation** for model selection (hyperparameter tuning).

For each case in the OOB dataset compute the OOB error:

1. Find all models that are not trained by the OOB instance.
2. Obtain prediction for each model
3. Average all these predictions (regression) or take the majority vote (classification) to compute the OOB error for each example, and average OOB error across examples.

Separating data

Original data, bootstrap data set, out-of-bag dataset



Random Forest (Breiman 01, Ho 98)

1. Parameters k and a ;
2. Get a dataset S of N labeled examples on A attributes;
3. Build k bagging replicas of S : $S_1, \dots, S_k$;
4. Use the k replicas to build k *random trees* $T_1, \dots, T_k$;
 1. At each node split, randomly select $a \leq A$ attributes, and choose best of these a ;
 2. Grow each tree as deep as possible: not pruning!!
5. Predict using majority vote of the T_i 's.

Random Forest

Weak learner strength vs. weak learner variance.

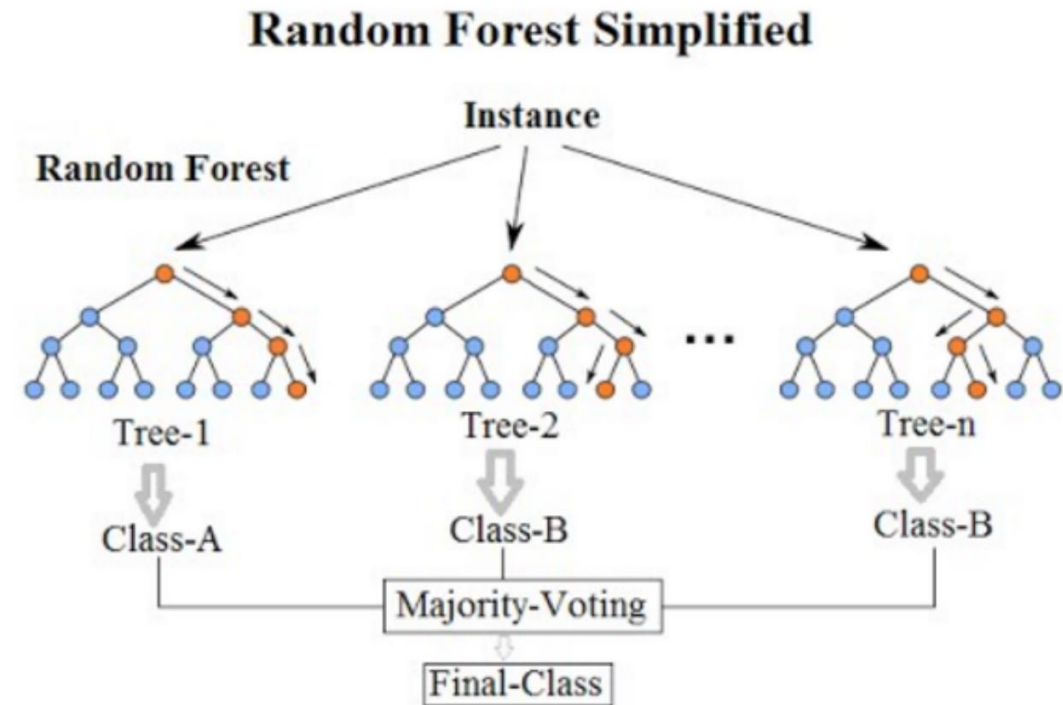
- More attributes a increases strength, overfits more.
- More trees k decreases variance, overfits less.

Can be shown to be similar to weighted k-NN.

Top performer in many tasks

Random Forest

- Builds in parallel n decision trees.
- Bootstrapping data.
- Feature bagging: at each candidate split in the learning process selects a random subset of features.



Random Forest

If a random forest contains many trees, it can be difficult to comprehend what the model is doing (not interpretable by a person).

- Variable importance plot add interpretability to the model.

Variable importance

1. Gini-based variable importance

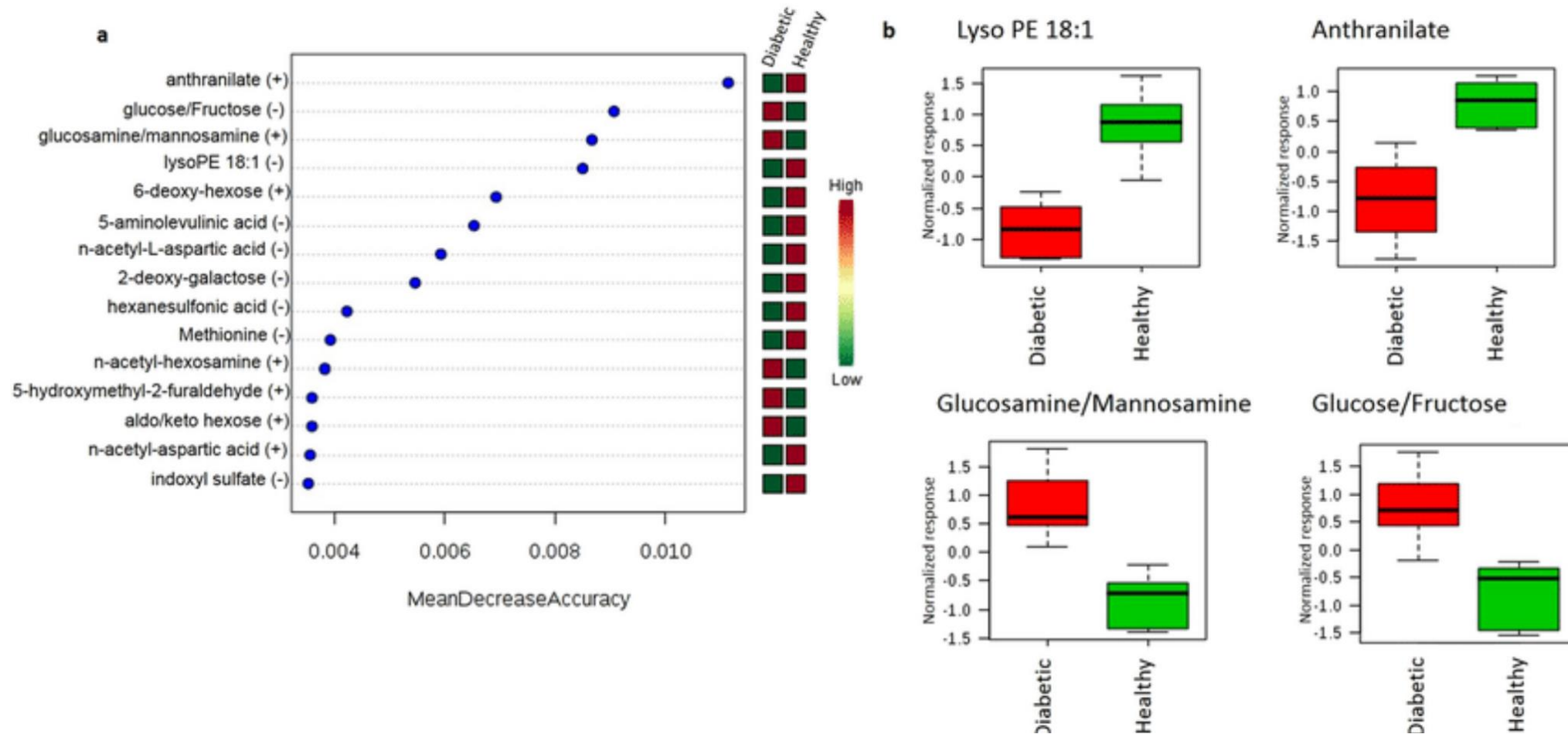
Add Gini impurity gains for variables in splits in each tree in the forest, sort variables by their sum.

2. Permutation-based variable importance

For each variable, permute values and compute difference in OOB error metrics before and after permutation. If variable is important, then accuracy in the permuted copy should decrease. Sort variables by this difference.

Permutation-based more reliable, but slower; Gini-based is biased towards categorical variables with many splits.

Feature importance plot



Boosting

Boosting involves sequentially training models, each trying to correct the errors of its predecessor. The final model is a weighted sum of the individual models.

Example: AdaBoost, Gradient Boosting, XGBoost.

Boosting

- Bagging tries to reduce variance of base classifiers by building different bootstrapping datasets.
- Boosting tries to actively improve accuracy of weak classifiers.
- How? By training a sequence of specialized classifiers based on previous errors .

Boosting I (Schapire 92)

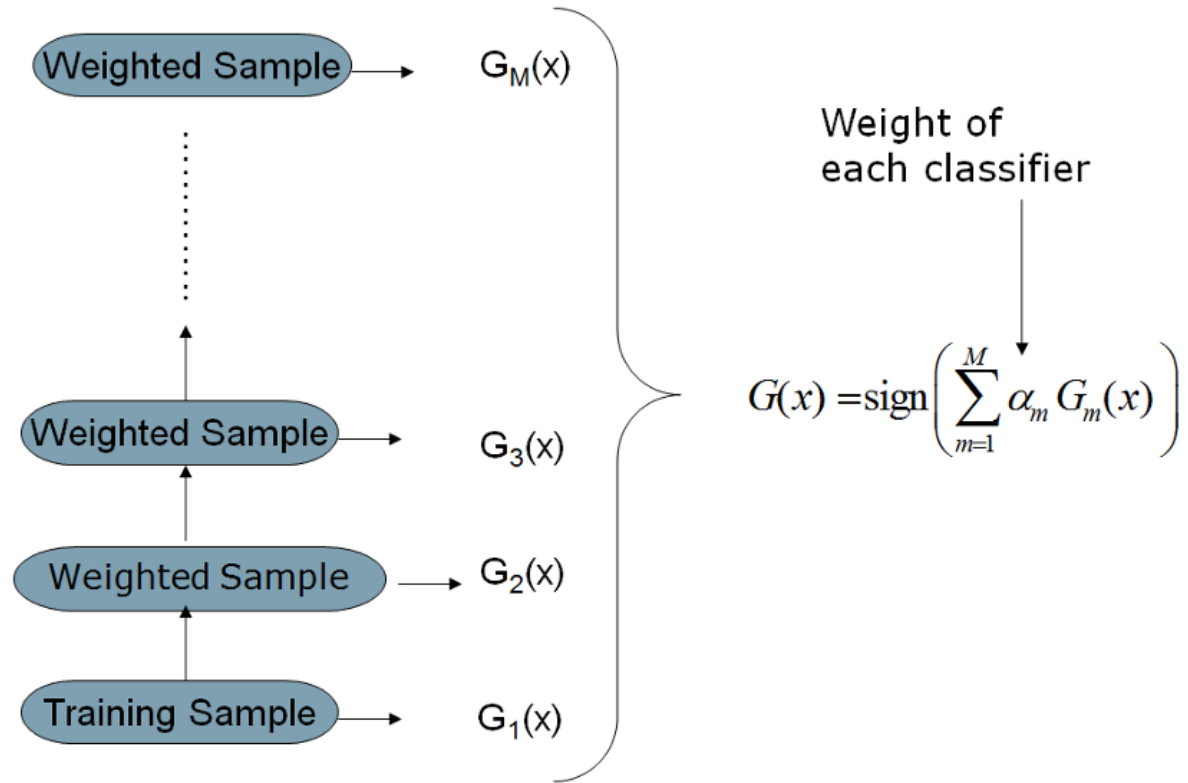
- Works on top of any weak learner. A weak learner is defined as any learning mechanism that works better than chance (accuracy > 0.5 when two equally probable classes)
- Adaptively, sequentially, creating classifiers.
- Classifiers and instances have varying weights.
- Increases the weight of incorrectly classified instances.
- Final label as weighting voting of sequence of classifiers.

Boosting

- ▶ Only two classes
- ▶ Output: $y \in \{-1, 1\}$
- ▶ Examples: X
- ▶ Weak Classifier: $G(X)$
- ▶ Error de training (err_{train})

$$err_{train} = \frac{1}{N} \sum_{i=1}^N I(y_i \neq G(x_i))$$

Boosting



Adaboost algorithm

Set weight of all examples to $1/n$

For $t=1:L$

S_t = training set using weights for each example

Learn $G_t(S_t)$

Compute err_t for G_t

Compute $\alpha_m = \frac{1}{2} \ln\left(\frac{1 - err_t}{err_t}\right)$

Compute new weights $w_i \leftarrow \frac{w_i}{Z_t} \cdot e^{-[\alpha_t \cdot y_i \cdot G_t(x_i)]}$

Return classifier: $G(x) = \text{signe}\left(\sum_{t=1}^L \alpha_m G_t(x)\right)$

Adaboost algorithm

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \text{err}_m}{\text{err}_m} \right) > 0$$

$$w_i \leftarrow \frac{w_i}{Z_m} \cdot e^{-[\alpha_m \cdot y_i \cdot G(x_i)]}$$

$$G(x) = \text{sign} \left(\sum_{m=1}^L \alpha_m G_m(x) \right)$$

Adaboost algorithm

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \text{err}_m}{\text{err}_m} \right) > 0$$

$$w_i \leftarrow \frac{w_i}{Z_m} \cdot \begin{cases} e^{-\alpha_t} & \text{if } y_i = G_t(x_i) \\ e^{\alpha_t} & \text{if } y_i \neq G_t(x_i) \end{cases}$$

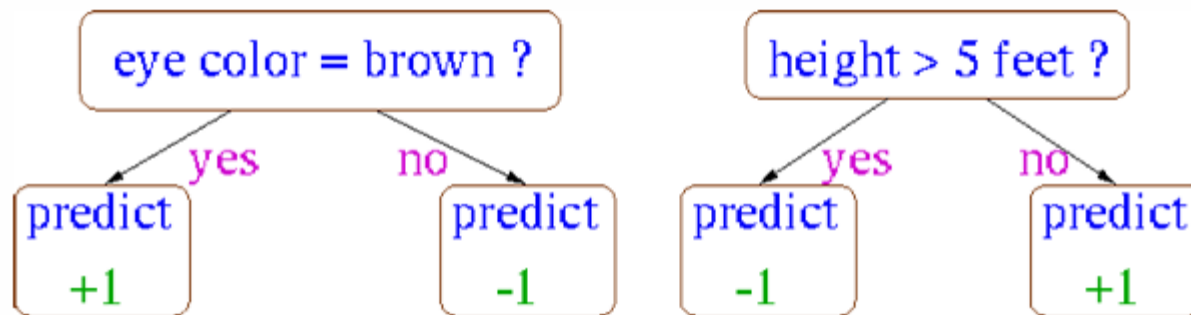
$$G(x) = \text{sign} \left(\sum_{m=1}^L \alpha_m G_m(x) \right)$$

Simple example

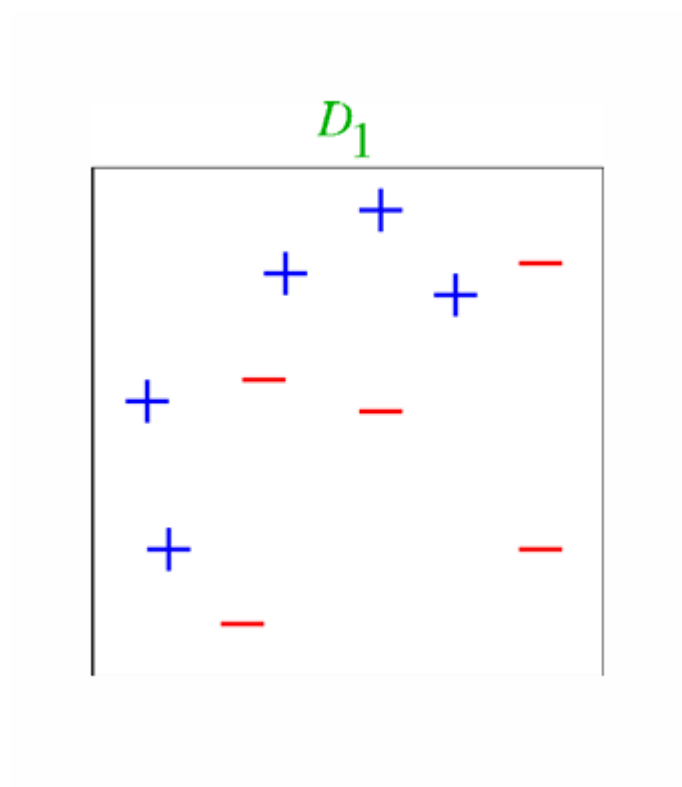
We will use Decision stumps as the weak learner Decision stumps are decision trees pruned to only one level.

Good candidates to weak learners: above 0.5 accuracy and high variance.

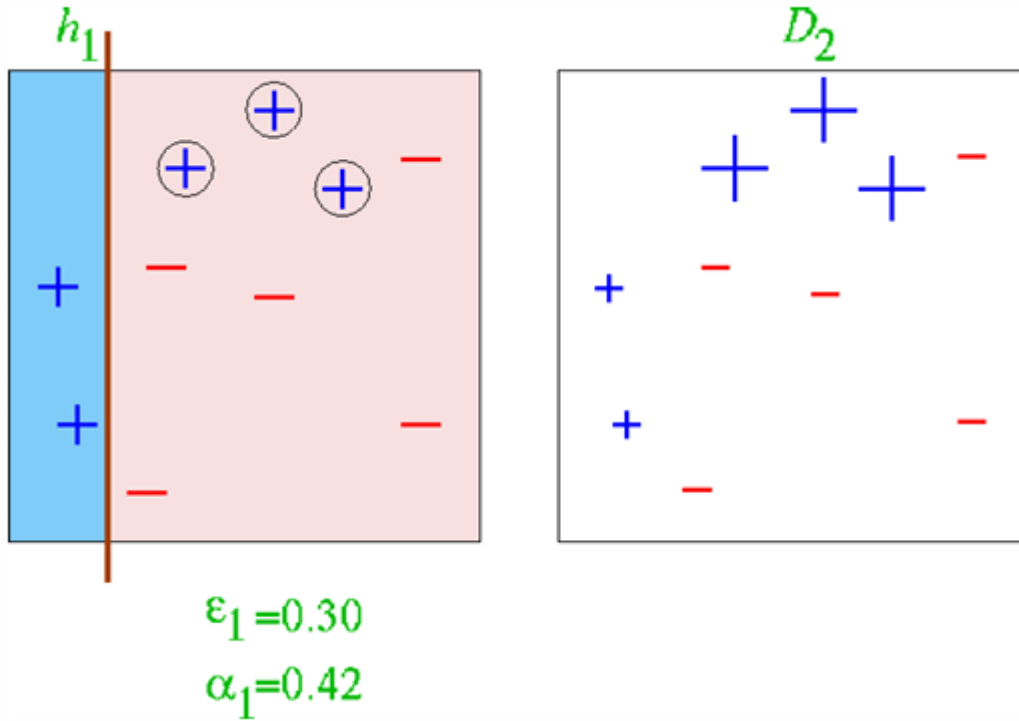
Two examples of decision stumps.



Simple examples

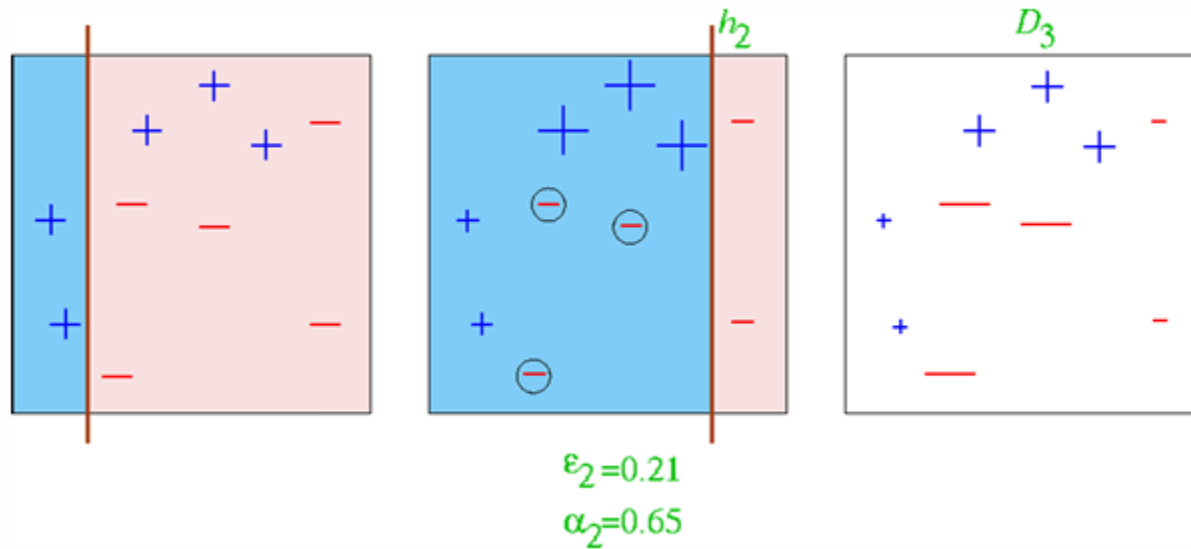


Simple examples



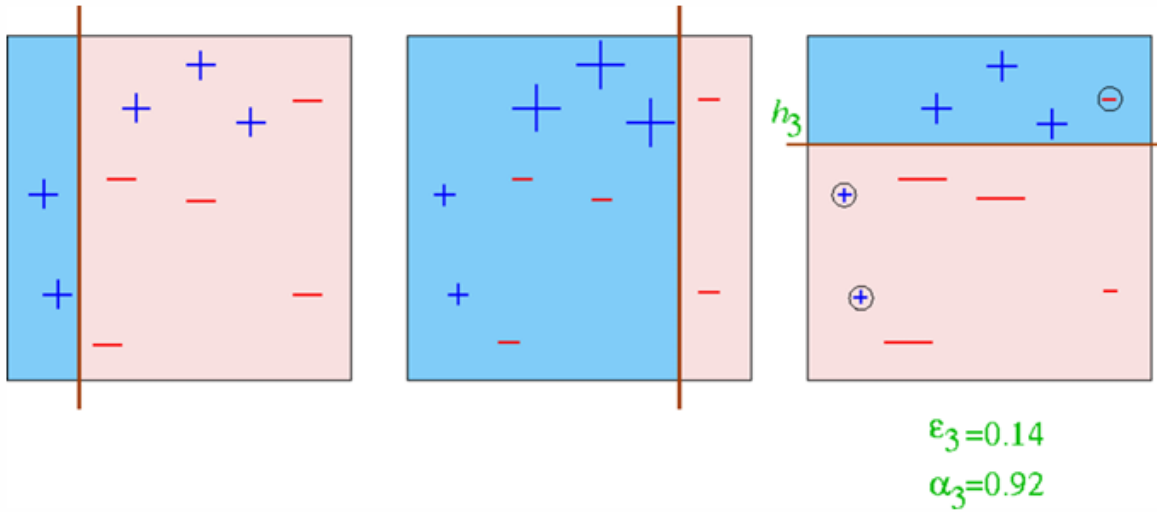
$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - err_m}{err_m} \right) > 0$$

Simple examples



$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - err_m}{err_m} \right) > 0$$

Simple examples

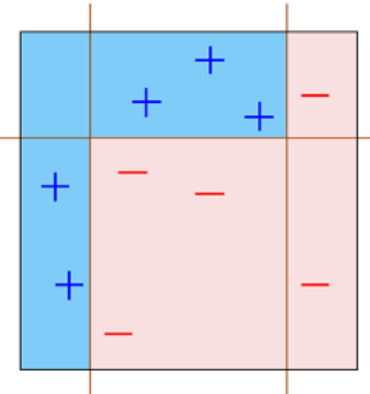


$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - err_m}{err_m} \right) > 0$$

Simple examples

$$H_{\text{final}} = \text{sign} \left(0.42 \begin{array}{|c|c|} \hline \text{blue} & \text{red} \\ \hline \end{array} + 0.65 \begin{array}{|c|c|} \hline \text{blue} & \text{blue} \\ \hline \end{array} + 0.92 \begin{array}{|c|c|} \hline \text{blue} & \text{red} \\ \hline \end{array} \right)$$

=

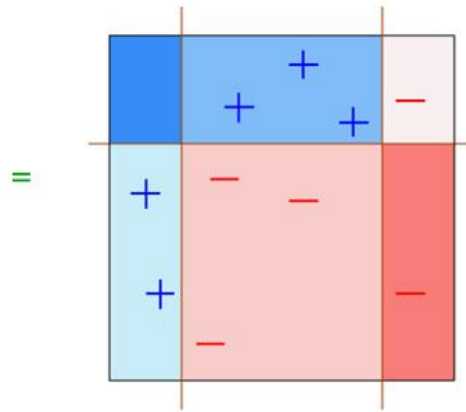


The diagram illustrates the final hypothesis H_{final} as a weighted sum of three weak hypotheses, each represented by a square with a vertical decision boundary. The weights are 0.42, 0.65, and 0.92. The first hypothesis has a blue region on the left and a red region on the right. The second hypothesis has a blue region on the left and a blue region on the right. The third hypothesis has a blue region on the left and a red region on the right. The final hypothesis is shown as a 2D plot with a horizontal and vertical decision boundary, resulting in four regions: top-left (blue, +), top-right (red, -), bottom-left (blue, +), and bottom-right (red, -).

Simple example

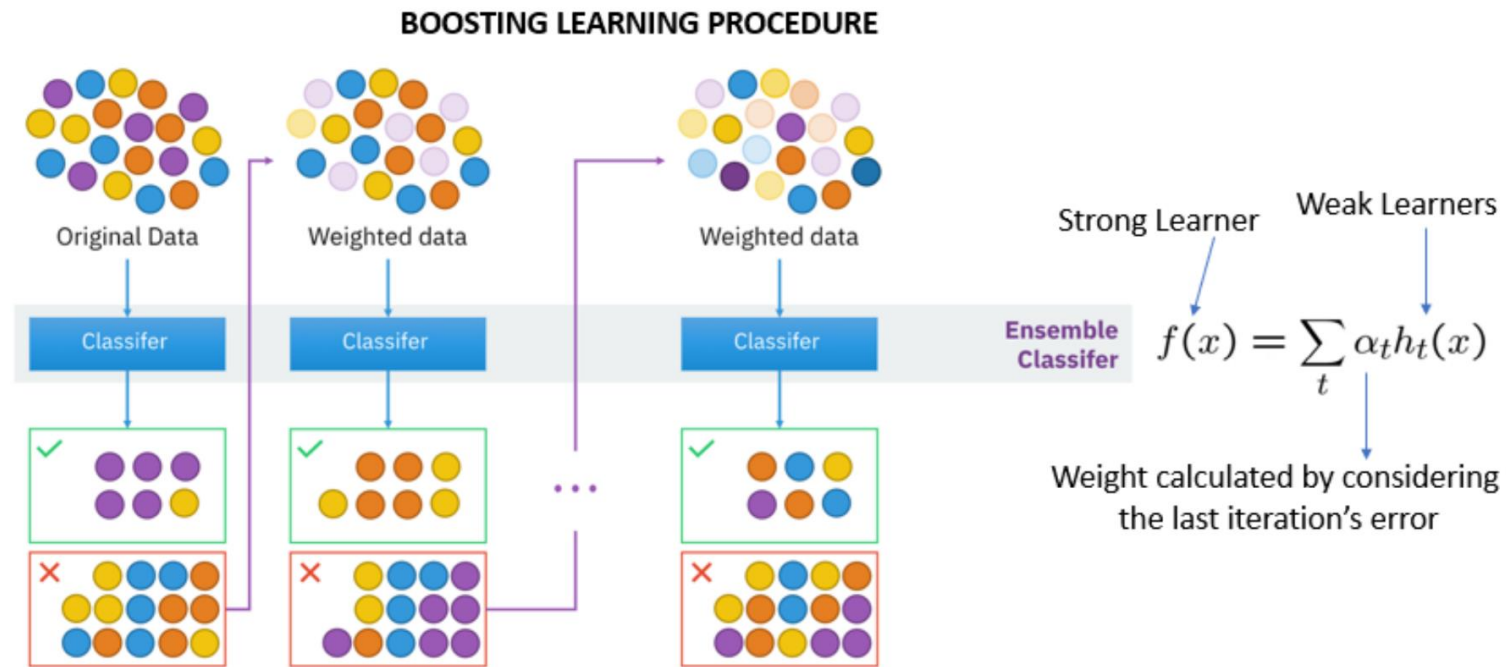
$$f = \left(\begin{array}{c} 0.42 \\ + 0.65 \\ + 0.92 \end{array} \right)$$

$/(0.42 + 0.65 + 0.92)$



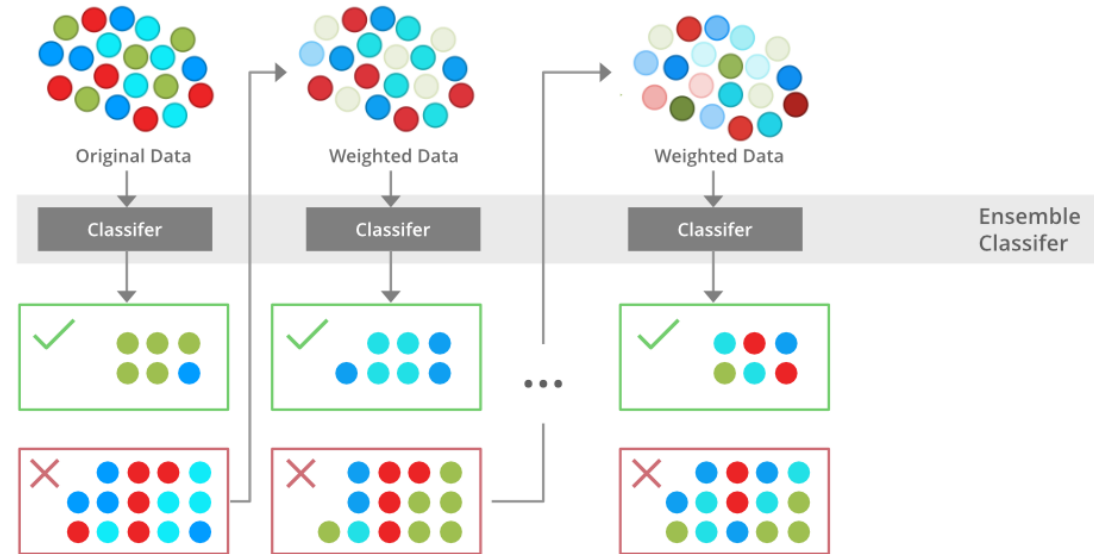
Boosting

Constructs a strong classifier from sequentially learning n weak classifiers.



Boosting

- Improved accuracy.
- Robustness: reweight instances which are misclassified.
- Robust to class imbalance.
- Too slow to be used in real-time applications.
- Computationally expensive.



Bagged Boosting (Hybrid Method)

- Combines bagging and boosting to leverage the advantages of both methods. This hybrid approach aims to reduce both variance and bias.
- **Example:** Gradient Boosted Decision Trees (GBDT) with bagging applied to each boosting iteration.

Summary

- **Bagging:** It is a homogeneous weak learners' model that learns from each other independently in parallel and combines them for determining the model average.
- **Boosting:** It is also a homogeneous weak learners' model but works differently from Bagging. In this model, learners learn sequentially and adaptively to improve model predictions of a learning algorithm.
- **Stacking:** Uses heterogenous classifiers.

Reasons for using ensemble models

1. Statistical reasons: We do not rely on one classifier, so we reduce variance.
2. Computational reasons: A weak classifier can be stuck in local minima. When starting from different training data sets, we can find better solution
3. Representational reasons: Combination of classifiers return solutions outside the initial set of hypothesis, so they adapt better to the problem.

Comparison

Bagging	Boosting
Aim to decrease variance, not bias.	Aim to decrease bias, not variance.
Each model is built independently.	New models are influenced by the performance of previously built models.
Different training data subsets are selected using row sampling methods from the entire training dataset.	Every new subset contains the elements that were misclassified by previous models.
Tries to solve the overfitting problem.	Tries to reduce bias.
If the classifier is unstable (high variance), then apply bagging.	If the classifier is stable and simple (high bias) then apply boosting.
Trained in parallel.	Base classifiers are trained sequentially.
Example: Random Forest	Example: Adaboost